The Globe oposiciones

Bloque 3 - Tema 6

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS

Resumen C++

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE 2

1. INTRODUCCIÓN 3
   1. *Palabras reservadas del lenguaje 3*
2. EXTENSIONES DE C 5
   1. *Comentarios 5*
   2. *Declaraciones de variables y visibilidad 5*
   3. *Operador de resolución de alcance unario 6*
   4. *Flujo de entrada/salida 6*
   5. *Nuevos tipos de datos 7*
   6. *Funciones en línea (inline) 7*
   7. *Parámetros por referencia 8*
   8. *Valores por defecto de parámetros en una función 8*
   9. *Sobrecarga de funciones 9*
   10. *Calificador const 9*
   11. *Asignación dinámica de memoria 10*
   12. *Plantillas de funciones 10*
3. PROGRAMACIÓN ORIENTADA A OBJETOS 11
   1. *Clases 11*
   2. *Asignación dinámica de memoria: new y delete 12*
   3. *Constructores de una clase 12*
   4. *Destructor 13*
   5. *Puntero this 14*
   6. *Funciones o clases amigo 14*
   7. *Sobrecarga de operadores 14*
4. HERENCIA 16
   1. *Tipos de herencia 16*
   2. *Clases base públicas, protegidas y privadas 17*
   3. *Clases abstractas 17*
5. PLANTILLAS 18

1. INTRODUCCIÓN

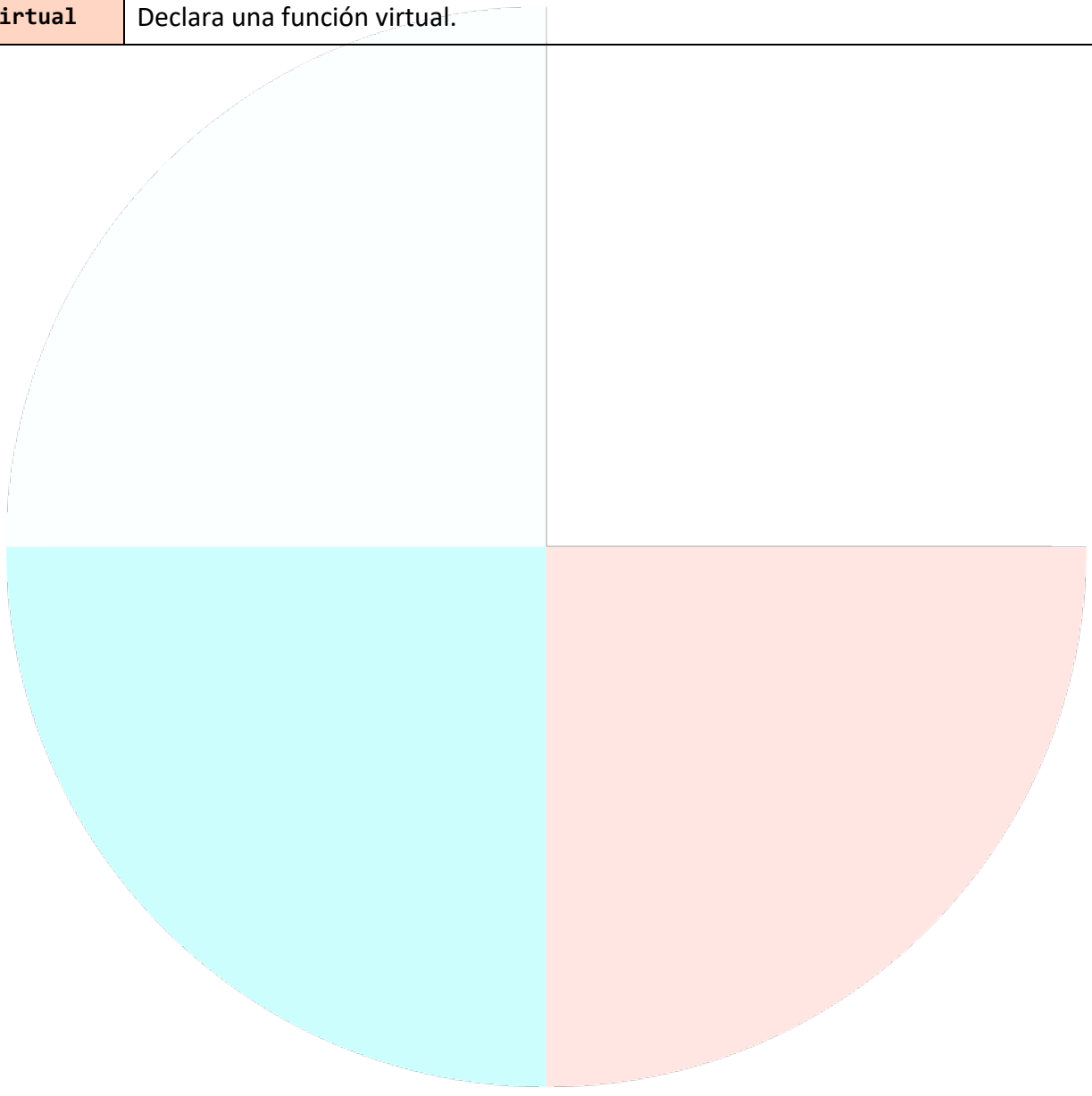
El lenguaje C++ se comenzó a desarrollar en 1980. Su autor fue B. Stroustrup. C++ es un lenguaje versátil, potente y general. Mantiene las ventajas del C en cuanto a riqueza de operadores y expresiones, flexibilidad, concisión y eficiencia. Además, ha eliminado algunas de las dificultades y limitaciones del C original

1. Palabras reservadas del lenguaje

Además de las palabras reservadas en C, que también son propias de C++, tenemos una serie de palabras reservadas exclusivas de C++:

asm	Medio definido por la puesta en práctica de lenguaje de ensamble a lo largo de C++.
bool	Tipo de dato para manejar datos lógicos: `true` y `false`.
catch	Maneja una excepción generada por un `throw`.
class	Define una nueva clase. Pueden crearse objetos de esta clase.
delete	Destruye un objeto de memoria creado con new.
false	Valor falso de un tipo de dato `bool`. Equivale a cero.
friend	Declara una función o una clase que sea un ''amigo'' de otra clase. Los amigos pueden tener acceso a todos los miembros de datos y a todas las funciones miembro de una clase.
inline	Avisa al compilador que una función particular deberá ser generada en línea, en vez de requerir de una llamada de función.
new	Asigna dinámicamente un objeto de memoria en la memoria adicional disponible para el programa en tiempo de ejecución. Determina automáticamente el tamaño del objeto.
operator	Declara un operador ''homónimo''.
private	Un miembro de clase accesible a funciones miembro y a funciones `friend` de la clase de miembros `private`.
protected	Una forma extendida de acceso `private`; también se puede tener acceso a los miembros `protected` por funciones miembro de clases derivadas y amigos de clases derivadas.
public	Un miembro de clase accesible a cualquier función.
template	Declara cómo construir una clase o una función, usando una variedad de tipos.
this	Un apuntador declarado en forma implícita en toda función de miembro no `static` de una clase. Señala al objeto al cual esta función miembro ha sido invocada.

throw	Transfiere control a un manejador de excepción o termina la ejecución del programa si no puede ser localizado un manejador apropiado.
true	Valor verdadero de un tipo de dato `bool`. Equivale a uno.
try	Crea un bloque que contiene un conjunto de números que pudieran generar excepciones, y habilita el manejo de excepciones para cualquier excepción generada.
virtual	Declara una función virtual.



2. EXTENSIONES DE C

1. Comentarios

En C los comentarios empiezan por los caracteres `/*` y terminan con los caracteres `*/`. Pueden comprender varias líneas y estar distribuidos de cualquier forma, pero todo aquello que está entre el `/*` (inicio del comentario) y el `*/` (fin del comentario) es simplemente ignorado por el compilador.

En C++ se admite el mismo tipo de comentarios que en C, pero además se considera que son comentarios todo aquel texto que está desde dos barras consecutivas `//` hasta el fin de la línea.

Ejemplo:

```
// Esto es un comentario simple.
// Esto es un comentario más largo,
// distribuido en varias líneas. El
// texto se suele indentar por la izquierda.
```

2. Declaraciones de variables y visibilidad

En C++, a diferencia de C, las declaraciones pueden ser colocadas en cualquier parte, siempre y cuando las declaraciones antecedan el uso de lo que se está declarando. También, las variables pueden ser declaradas en la sección de inicialización de una estructura `for`; el alcance de dichas variables se mantiene hasta el final del bloque en el cual la estructura for está definida.

El alcance de una variable local C++ empieza en su declaración y se extiende hasta la llave derecha de cierre. Las declaraciones de variables no pueden ser colocadas en la condición de una estructura `while`, `do/while`, `for` o `if`.

Ejemplo:

```
for (double suma = 0.0, int i = 0; i<n; i++)
   suma += a[i];
```

El alcance de una variable es conocido como **visibilidad de una variable**: parte del programa en la que esa variable está definida y puede ser utilizada.

Las variables locales se crean dentro de un bloque y sólo son visibles dentro del bloque en el que han sido definidas y en sus bloques anidados, salvo que sean ocultadas por una nueva variable del mismo nombre declarada en uno de esos bloques anidados. Una variable local declarada dentro de un bloque oculta una variable global del mismo nombre.

Ejemplo:

```
{
	double x = 2.0;
	printf("valor=", x); // se imprime 2.0
	{
		printf("valor", x); // se imprime 2.0
		double x = 3.0;
		printf("valor", x); // se imprime 3.0
	}
	printf("valor", x); // se imprime 2.0
}
```

3. Operador de resolución de alcance unario

C++ dispone del operador (::), llamado operador de resolución de alcance unario. Este operador, antepuesto al nombre de una variable global que está oculta por una variable local del mismo nombre, permite acceder al valor de la variable global (no válido entre variables locales).

Ejemplo:

```
#include <iostream.h>
float v;
int main( )
{
	int v = 7;
	::v = 10.5; // Utilizar la variable global v
	cout << "variable local v = " << v << "\n";
	cout << "variable global v = " << ::v << "\n";
	return 0;
}
```

4. Flujo de entrada/salida

C++ ofrece una alternativa a las llamadas de función `printf` y `scanf` para manejar la entrada/salida de los tipos y cadenas de datos estándar.

Así, en lugar de `printf` usamos el flujo estándar de salida `cout` y el operador `<<` (''colocar en'').

Y en vez de `scanf` usamos el flujo de entrada estándar `cin` y el operador `>>` (''obtener de'').

Estos operadores de inserción y extracción de flujo, a diferencia de `printf` y `scanf`, no requieren de cadenas de formato y de especificadores de conversión para indicar los tipos de datos que son extraídos o introducidos.

Para utilizar entradas/salidas de flujo, se debe incluir el archivo de cabecera `iostream.h`.

Ejemplo:

```
char nombre;
int num=2;
cout << "Introduzca el nombre del fichero " << num << ": ";
cin >> nombre;
```

Para generar un salto de línea utilizamos `endl`.

```
cout << "El resultado es " << num << endl;
```

5. Nuevos tipos de datos

A diferencia de C, en C++ existe el tipo de datos booleano `bool`, que admite como valores las palabras reservadas `true` y `false`.

C++ proporciona la capacidad de crear tipos definidos por el usuario mediante el uso de las palabras reservadas `enum`, `struct`, `union` y la nueva palabra reservada `class`. A diferencia de C, una enumeración en C++, cuando se declara, se convierte en un tipo nuevo. Para declarar la variable del nuevo tipo, la palabra reservada `enum` no es requerida. Lo mismo para `struct`, `union` y `class`.

Ejemplo:

```
struct alumno {
long nmat;
char nombre[41];
};

alumno delegado; // esto es C++
struct alumno subdelegado; /* esto es C */
```

6. Funciones en línea (inline)

C++ permite sustituir, en tiempo de compilación, la llamada a una función por el código correspondiente en el punto en que se realiza la llamada. De esta manera la ejecución es más rápida, pues no se pierde tiempo transfiriendo el control y realizando conversiones de parámetros. Está pensado para funciones pequeñas. Se utiliza la palabra reservada `inline` y la sintaxis es:

```
inline tipo función(parámetros) {sentencias}
```

Ejemplo:

```
#include <iostream.h>
inline float cubo( const float s ) { return s * s * s; }
main( ) {
cout << "Introduce la longitud del lado de tu cubo: ";
float lado;
cin >> lado;
cout << "El volumen del cubo de lado " << lado << "es" << cubo(lado) << "\n";
return 0; }
```

7. Parámetros por referencia

C++ ofrece una nueva forma de pasar argumentos por referencia a una función, que no obliga a utilizar (dentro de la función) el operador indirección (*) para acceder al valor de la variable que se quiere modificar. Esto se hace por medio de un nuevo tipo de dato (que no existe en C) llamado tipo referencia (reference).

Para indicar que un parámetro de función es pasado por referencia, sólo hay que colocar un ampersand (&) después del tipo del parámetro en el prototipo de función. Usamos apuntadores para pasar argumentos que pudieran ser modificados por la función llamada, y usar referencias a constantes para pasar argumentos extensos, que no serán modificados.

Ejemplo:

C	C++

`#include <stdio.h>`
`#include <stdlib.h>`
`void main(void)`
`{`
`int i = 1, j = 2;`
`printf("\ni = %d, j = %d", i, j);`
`permutar(&i, &j);`
`printf("\ni = %d, j = %d", i, j);`
`}`
`void permutar(int *a, int *b)`
`{`
`int temp;`
`temp = *a;`
`*a = *b;`
`*b = temp;`
`}`
	`#include <stdio.h>`
	`#include <stdlib.h>`
	`void main(void)`
	`{`
	`int i = 1, j = 2;`
	`printf("\ni = %d, j = %d", i, j);`
	`permutar(i, j); // los argumentos no llevan (*) ni (&)`
	`printf("\ni = %d, j = %d", i, j);`
	`}`
	`void permutar(int &a, int &b) // los argumentos son referencias`
	`{`
	`int temp;`
	`temp = a; // no hace falta utilizar`
	`a = b; // el operador indirección (*)`
	`b = temp;`
	`}`

8. Valores por defecto de parámetros en una función

En C++ se pueden definir valores por defecto para todos o algunos de los parámetros de una función. Así, cuando se llama a una función, en el caso de que algún argumento esté ausente de la lista de argumentos actuales, se toma el valor asignado por defecto a ese argumento.

Ejemplo: `double modulo(double x[], int n=3);`

```
v = modulo(x, n);
v = modulo(x); // Se utiliza el valor por defecto n=3
```

En C++ se exige que todos los argumentos con valores por defecto estén al final de la lista de argumentos. En la llamada a la función pueden omitirse alguno o algunos de los últimos argumentos de la lista. Si se omite un argumento deben de omitirse todos aquellos que se encuentren detrás suyo.

9. Sobrecarga de funciones

La sobrecarga de funciones consiste en declarar y definir varias funciones distintas que tienen un mismo nombre, pero un número y tipo diferente de parámetros (el tipo de retorno no influye). Este concepto también es conocido como homonimia de funciones.

Ejemplo:

```
#include <stdio.h>
int max( int a, int b )
{
	if ( a > b ) return a;
	return b;
}
char *max( char *a, char *b )
{
	if ( strcmp (a,b) > 0 ) return a;
	return b;
}
int main( )
{
	printf ( ''max( 19, 69 ) = %d\n'', max( 19, 69 ) );
	printf ( ''max( abc,def ) = %s\n'', max( ''abc'',''def'' ) );
	return 0;
}
```

10. Calificador const

	DECLARACIÓN	SIGNIFICADO	EXPLICACIÓN
1	`const tipo nombre = valor;`	Variable constante	No se puede modificar el valor de la variable
2	`const tipo *nombre = valor;`	Puntero variable apuntando a variable constante	No se puede modificar el valor de la variable
3	`tipo *const nombre = valor;`	Puntero constante apuntando a variable modificable	No se puede modificar la dirección a la que apunta, sí el valor
4	`const tipo *const nombre = valor;`	Puntero constante apuntando a variable constante	No se pueden modificar ni la dirección a la que apunta ni el valor de la variable

Usos y ejemplos:

(1) Definir variables constantes: `const int SIZE = 5;`

(2) Especificar que un parámetro de una función no es modificable:

```
char *strcpy(char *s1, const char *s2);
```

(3) Definir un puntero constante: `char * const s2;`

11. Asignación dinámica de memoria

Con los operadores **`new`** y **`delete`** el programador tiene entera libertad para decidir crear o destruir sus variables cuando las necesite. Una variable creada con el operador `new` dentro de cualquier bloque, perdura hasta que es explícitamente borrada con el operador `delete`. Puede traspasar la frontera de su bloque y ser manipulada por instrucciones de otros bloques.

```
ptr = new typeName;
```

Asigna memoria para un objeto del tipo `typeName`. El operador new crea automáticamente un objeto del tamaño apropiado, y devuelve un puntero (`ptr`) del tipo apropiado. Si mediante `new` no se puede asignar memoria, devuelve un puntero nulo.

```
delete ptr;
```

Se usa para liberar el espacio para este objeto. El operador `delete` sólo puede ser utilizado para desasignar memoria ya asignada mediante el operador `new`.

12. Plantillas de funciones

Las plantillas de función permiten la creación de funciones que ejecuten las mismas operaciones sobre distintos tipos de datos, pero la plantilla de función se define sólo una vez.

Las plantillas de función proporcionan, como las macros, una solución compacta, pero permiten verificación completa de tipo.

Todas las definiciones de plantillas de función empiezan con la palabra reservada `template`.

3. PROGRAMACIÓN ORIENTADA A OBJETOS

1. Clases

Las clases permiten que el programador modele objetos que tienen atributos (variables miembro) y comportamientos u operaciones (funciones miembro). Son definidos mediante la palabra reservada **class**.

El nombre de la clase (una vez definida) puede ser utilizado para declarar objetos de dicha clase.

Ejemplo:

```
class Punto // Declaración de la clase Punto.
{
      int _x, _y; // Coordenadas del punto.
public: // Principio de la declaración de interface.
      void setX (const int val);
      void setY (const int val);
      int getX( ) {return _x;}
      int getY( ) {return _y;}
};
Punto apunto; // Definición del objeto apunto.
```

La definición de una clase comienza con la palabra reservada `class` . El cuerpo de la definición de clase se delimita mediante llaves. La definición de clase termina con un punto y coma. En el cuerpo de la definición existen partes nuevas: la etiqueta `public:` y `private:` se conocen como especificadores de acceso de miembro. Cualquier atributo o función miembro declarado después del especificador de acceso de miembro `public:` (y antes del siguiente especificador de acceso de miembro) es accesible, siempre que el programa tenga acceso a un objeto de la clase. Cualquier atributo o función miembro declarada después del especificador de acceso de miembro `private:` (y hasta el siguiente especificador de acceso de miembro) sólo es accesible a las funciones miembro de la clase.

Estos especificadores pueden aparecer varias veces en una definición de clase. Por defecto, los elementos de las clases son privados (`private:`).

Normalmente, los atributos aparecen listados en la parte `private:` de una clase, y normalmente, las funciones de miembro aparecen listadas en la porción `public:`, aunque puede darse el caso contrario.

Los atributos de una clase no pueden ser inicializados donde son declarados en el cuerpo de la clase. Deberán ser inicializados por el constructor de la clase, o las funciones ''set'' les podrán asignar valores.

Si nos fijamos en el ejemplo, vemos que las funciones miembro `getX` y getY están definidos, sin embargo, no ocurre lo mismo con `setX` y `setY`. Entonces, cuando una función miembro se define fuera de la definición de clase, el nombre de función es antecedido por el nombre de

la clase y por el operador de resolución de alcance binario (::). Dicha función miembro queda dentro del alcance de la clase, es decir, es como si se hubiera definido cuando se definió la clase.

Ejemplo:

```
void Punto :: setX (const int val)
{ _x = val; }
void Punto :: setY (const int val)
{ _y = val; }
```

Cada objeto de una clase tiene su propia copia de cada una de las variables miembro de esa clase. Sin embargo, a veces puede interesar que una variable miembro sea común para todos los objetos de la clase, de modo que todos compartan el mismo valor. En tal caso se puede utilizar una **variable miembro `static`**.

Ejemplo: `static float pi;`

Las variables `static` pueden ser `public` o `private`, del mismo modo que el resto de las variables miembro. Las variables `static` de una clase existen aunque no se haya declarado ningún objeto de esa clase. Esto quiere decir que la memoria reservada para este tipo de variable se ocupa en el momento de la definición de la clase, no en el momento de la declaración de los objetos.

2. Asignación dinámica de memoria: new y delete

Los operadores `new` y `delete` pueden ser aplicados tanto a variables de los tipos predefinidos (`int`, `float`, `double`…) como a objetos de las clases definidas por el usuario. Su principal aplicación está en la creación de variables con reserva dinámica de memoria.

Ejemplo: `Punto p = new Punto();`

3. Constructores de una clase

C++ no permite crear objetos sin dar un valor inicial apropiado a todas sus variables miembro. Para ello, se utilizan los **constructores**. Estos son unas funciones miembro de una clase especiales que se llaman de modo automático al crear los objetos, y dan un valor inicial a cada una de las variables miembro.

El nombre del constructor es siempre el mismo que el nombre de la clase. Además, los constructores se caracterizan porque se declaran y definen sin valor de retorno, ni siquiera `void`. El **constructor por defecto** es aquel al que no tiene parámetros o que si los tiene, todos los parámetros tienen un valor por defecto.

Ejemplo: `Punto() { _x = _y = 0;}`

```
Punto(const int x=0, const int y=0) {_x = x; _y = y;}
```

El uso del constructor es tan importante que, en el caso de que el programador no defina ningún constructor para una clase, el compilador de C++ proporciona un constructor por defecto.

Utilizando las capacidades de sobrecarga de funciones de C++, para una clase se pueden definir varios constructores.

Ejemplo: `Punto(const int x, const int y) {_x = x; _y = y;}`

Vistos el constructor por defecto y el resto de constructores, existe una forma más de inicializar los atributos o variables miembro.

Ejemplo: `Punto(const int x, const int y): _x(x), _y(y) {}`

Los **inicializadores** se introducen, tras el carácter dos puntos (**:**), separados por comas, justo antes de abrir las llaves del cuerpo del constructor. Constan del nombre de la variable miembro seguido, entre paréntesis, del argumento que le da valor. Los inicializadores son más eficientes que las sentencias de asignación, y además permiten definir variables miembro `const`, que pueden ser inicializadas pero no asignadas.

Un constructor especial es el **constructor de copia**. El objetivo es crear un objeto inicializándolo a partir de otro objeto de la misma clase. Tiene un único parámetro: una referencia constante a un objetivo de la clase.

Ejemplo: `Punto(const Punto &p) {_x=p.getX(); _y=p.getY(); }`

Otra forma de copiar los valores de un objeto en otro es sobrecargar el **operador asignación (=)**.

4. Destructor

Un **destructor** de clase es llamado automáticamente cuando un objeto de una clase se sale de alcance: debemos asegurarnos de que la memoria asignada sea liberada. El destructor es siempre único (no puede estar sobrecargado) y no tiene argumentos en ningún caso. Tampoco tiene valor de retorno. Su nombre es el mismo que el de la clase precedido por el carácter tilde (~).

Ejemplo: declaración `~Punto();`

5. Puntero this

Por último, cada objeto mantiene un puntero a sí mismo denominado **`this`** que es un argumento implícito en todas las referencias a miembros incluidos dentro de dicho objeto. El apuntador `this` puede también ser utilizado de forma explícita. Mediante el uso de la palabra reservada `this`, cada objeto puede determinar su propia dirección.

Ejemplo: `void setX(const int val){this->_x = val;}`

6. Funciones o clases amigo

Podemos definir que **funciones o clases sean amigos** de una clase para permitirles acceso directo a sus miembros de datos privados. Se puede declarar una función o toda una clase como un `friend` de otra clase.

Para declarar una función como un `friend` de una clase, en la definición de clase hay que preceder el prototipo de función con la palabra reservada `friend`. Las declaraciones de amistad pueden ser colocadas en cualquier parte de la definición de clase.

Ejemplo: `friend void setX (const int val);`

7. Sobrecarga de operadores

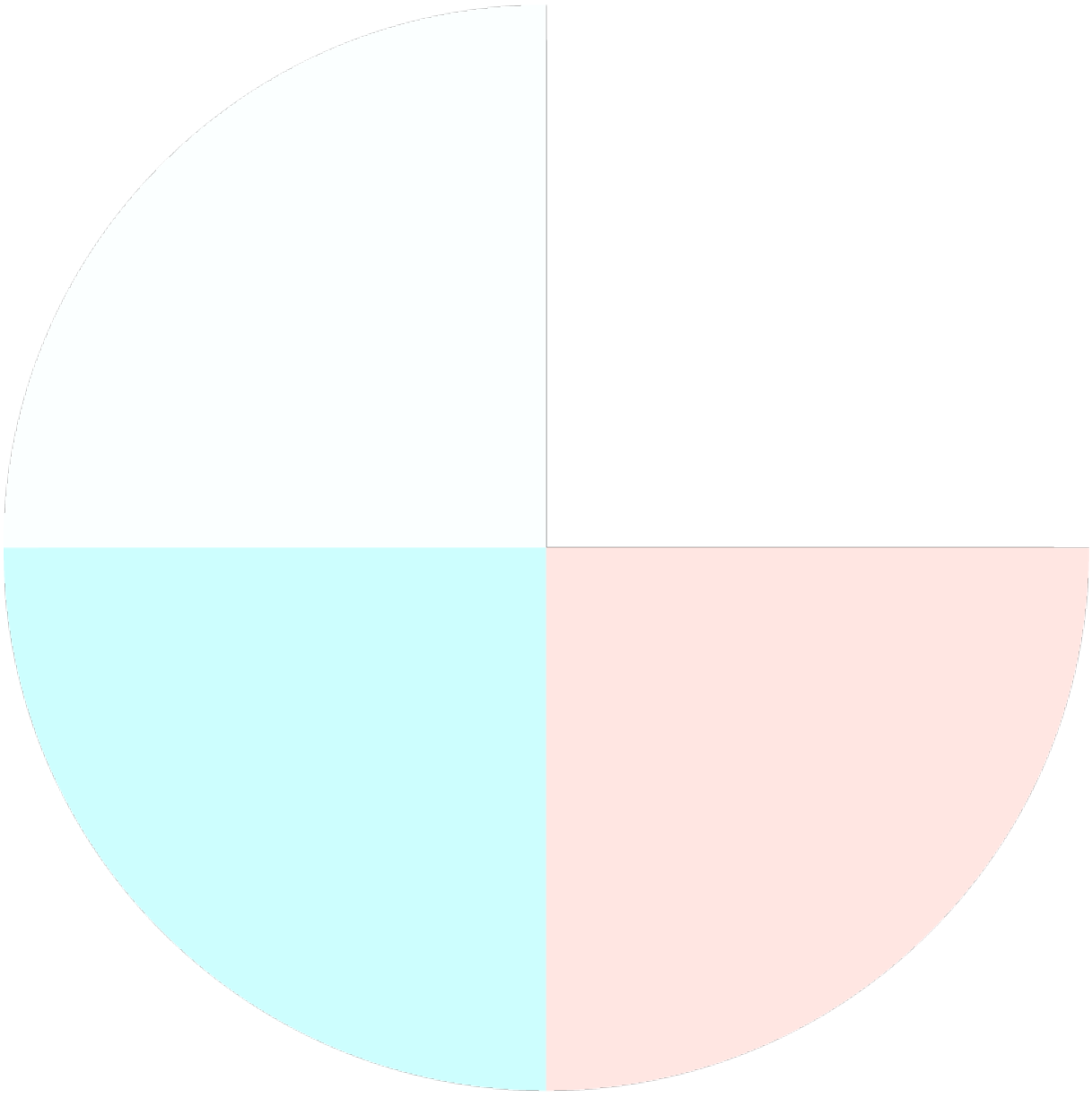
Los operadores de C++, al igual que las funciones, pueden ser sobrecargados. La sobrecarga de operadores quiere decir que se pueden redefinir algunos de los operadores existentes en C++ para que actúen de una determinada manera, definida por el programador, con los objetos de una clase determinada.

Ejemplo:

```
class cadena {
private:
	char* pstr;
	int nchar; // nº de caracteres (sin el '\0')
public:
	cadena(); // constructor por defecto
	cadena(char*); // constructor general
	cadena(const cadena&); // constructor de copia
	~cadena(); // destructor

	// sobrecarga de operadores
	cadena& operator= (const cadena&);
	friend cadena operator+ (const cadena&, const cadena&);
	friend cadena operator+ (const cadena&, const char* );
	friend cadena operator+ (const char*, const cadena&);
	friend int operator== (const cadena&, const cadena&);
	friend int operator!= (const cadena&, const cadena&);
	friend ostream& operator<< (ostream&, const cadena&);
```

```
        /* Otros operadores que se podrían sobrecargar:
        friend int operator== (const cadena&, const char*);
        friend int operator== (const char*, const cadena&);
        friend int operator!= (const cadena&, const char*);
        friend int operator!= (const char*, const cadena&); */
};
```



4. HERENCIA

Mediante la herencia única, una clase es derivada de una única clase base. Con la herencia múltiple una clase derivada hereda de múltiples clases base y C++ admite la herencia múltiple.

En C++, ''hereda de'' se reemplaza por dos puntos (**:**). La sintaxis para escribir una clase derivada es:

```
class tipo_clase_derivada: (public/private/protected) tipo_clase_base { };
```

Ejemplo:

```
class Puntos3D : public Punto
{
	int _z;
public:
	Puntos3D ( )
{
	setX(0);
	setY(0);
	_z = 0;
}
Puntos3D (const int x, const int y, const int z)
{
	setX(x);
	setY(y);
	_z = z;
}
	~Puntos3D( ) { /* Nada que hacer */}
	int getZ( ) { return _z;}
	void setZ ( const int val) { _z = val;}
};
```

1. Tipos de herencia

C++ distingue dos tipos de herencia: pública y privada. Por defecto, las clases se derivan unas de otras en forma privada. Si queremos herencia pública, debemos decírselo explícitamente al compilador.

El tipo de herencia influye sobre los privilegios de acceso a elementos de las diversas superclases.

Los miembros públicos de una clase base son accesibles a todas las funciones en el programa. Los miembros privados de clase base son accesibles sólo para las funciones miembro y los amigos de la clase base.

El tipo `protected` se usa para elementos que deberían ser usados directamente en las subclases, pero que no deberían estar accesibles desde fuera. Luego, el acceso protegido sirve como nivel intermedio de protección entre el acceso público y el privado. Los miembros

protegidos de clase base son accesibles sólo por miembros y amigos de la clase base, y por miembros y amigos de las clases derivadas.

Los miembros de clases derivadas pueden hacer referencia a miembros públicos y protegidos de la clase base sólo utilizando los nombres de los miembros.

2. Clases base públicas, protegidas y privadas

Al derivar una clase a partir de una clase base, la clase base puede ser heredada como `public`, `protected` o `private`.

Al derivar una clase a partir de una clase base pública, los miembros públicos de la base se convierten en miembros públicos de la clase derivada, y los miembros protegidos de la clase base se convierten en miembros protegidos de la clase derivada.

Los miembros privados de una clase base nunca son accesibles en forma directa desde una clase derivada.

Al derivar una clase a partir de una clase base protegida, los miembros públicos y protegidos de la clase base se convierten en miembros protegidos de la clase derivada.

Cuando se deriva una clase a partir de una clase base privada, los miembros públicos y protegidos de la clase base se convierten en miembros privados de la clase derivada.

Especificador de acceso	Accesible desde su propia clase	Accesible desde una clase derivada	Accesible desde fuera de la clase
public	SI	SI	SI
protected	SI	SI	NO
private	SI	NO	NO

3. Clases abstractas

Se denomina clase abstracta a aquella que contiene una o más funciones virtuales puras. El nombre proviene de que no puede existir ningún objeto de esa clase. Si una clase derivada no redefine una función virtual pura, la clase derivada la hereda como función virtual pura y se convierte también en clase abstracta. Por el contrario, aquellas clases derivadas que redefinen todas las funciones virtuales puras de sus clases base reciben el nombre de clases derivadas concretas, nomenclatura únicamente utilizada para diferenciarlas de las antes mencionadas.

Ejemplo: la clase Figura es una clase abstracta con una función virtual pura: area. Las clases derivadas de Figura, Cuadrado, Circulo y Elipse, redefinen esa función virtual.

5. PLANTILLAS

En C++, los tipos de datos genéricos son llamados plantillas de clase o simplemente plantillas (**templates**). Una plantilla de clase se parece a la definición de una clase normal, en la que algunos aspectos son representados por sustitutos.

Las definiciones de clases plantilla empiezan con la línea: `template <class T>` en la línea que antecede a la definición de clase. Puede existir más de un tipo parametrizado. Si es así, estarán separados por comas y cada tipo estará precedido por la palabra reservada `class`.

Ejemplo:

```
// fichero Pila.h
template <class T>
// declaración de la clase
class Pila
{
public:
	Pila(int nelem=10); // constructor
	void Poner(T);
	void Imprimir();
private:
	int nelementos;
	T* cadena;
	int limite;
};
```

```
// definición del constructor
template <class T> Pila<T>::Pila(int nelem)
{
	nelementos = nelem;
	cadena = new T(nelementos);
	limite = 0;
};

// definición de las funciones miembro
template <class T> void Pila<T>::Poner(T elem)
{
	if (limite < nelementos)
		cadena[limite++] = elem;
};

template <class T> void Pila<T>::Imprimir()
{
	int i;
	for (i=0; i<limite; i++)
		cout << cadena[i] << endl;
};
```

```
#include <iostream.h>
#include "Pila.h"
void main()
{
	Pila <int> p1(6);
	p1.Poner(2);
	p1.Poner(4);
	p1.Imprimir();
	Pila <char> p2(6);
	p2.Poner('a');
	p2.Poner('b');
	p2.Imprimir();
}
```

